CFD Project 1

Ben Rees

February 23, 2026

1 Introduction

In this project, the parabolic equation with boundary conditions as listed below is solved analytically and via a numerical discretization scheme. The results are compared and discussed.

Parabolic Equation:

$$\frac{\partial u}{\partial x} - 2\frac{\partial^2 u}{\partial y^2} = 2.$$

Boundary and Initial Conditions:

$$u(x,0) = 0, \quad u(x,1) = 0, \quad u(0,y) = 0.$$

2 Analytical Solution Derivation

The governing equation $\frac{\partial u}{\partial x} - 2\frac{\partial^2 u}{\partial y^2} = 2$ is linear but inhomogeneous. Using the principle of superposition, we define $u(x,y) = v(x,y) + f(y)$, where $f(y)$ is the steady-state solution and $v(x,y)$ is the transient homogeneous component.

2.1 Steady-State Solution

To homogenize the PDE, we require $f(y)$ to satisfy $-2f''(y) = 2$, or $f''(y) = -1$. Applying boundary conditions $f(0) = f(1) = 0$:

$$f(y) = -\frac{y^2}{2} + C_1 y + C_2 \xrightarrow{BCs} f(y) = \frac{y}{2}(1-y).$$

2.2 Transient Solution via Separation of Variables

The remaining homogeneous equation $\frac{\partial v}{\partial x} - 2\frac{\partial^2 v}{\partial y^2} = 0$ is solved assuming $v(x,y) = X(x)Y(y)$. Separating variables yields:

$$\frac{X'}{2X} = \frac{Y''}{Y} = -\lambda^2 \implies \begin{cases} Y'' + \lambda^2 Y = 0, & Y(0) = Y(1) = 0 \\ X' + 2\lambda^2 X = 0 \end{cases}.$$

Solving the eigenvalue problem gives $\lambda_n = n\pi$ and eigenfunctions $Y_n(y) = \sin(n\pi y)$ for $n = 1, 2, 3...$ The temporal solution is $X_n(x) = e^{-2(n\pi)^2 x}$. By superposition:

$$v(x, y) = \sum_{n=1}^{\infty} B_n \sin(n\pi y) e^{-2(n\pi)^2 x}.$$

Using the initial condition $u(0, y) = 0 \implies v(0, y) = -f(y)$, the Fourier coefficients B_n are calculated via orthogonality:

$$B_n = 2\int_0^1 -\frac{y}{2}(1-y)\sin(n\pi y)dy = \frac{2}{(n\pi)^3}[(-1)^n - 1].$$

The final analytical solution is $u(x, y) = \frac{y}{2}(1-y) + \sum_{n=1}^{\infty} B_n \sin(n\pi y) e^{-2(n\pi)^2 x}$.

3 Numerical Discretization and Implementation

In this report, a finite volume method based on the central difference approximation with a Crank-Nicolson time-stepping scheme is employed due to this scheme's second-order accuracy, stability, and conservativeness. The spatial domain (1D) is discretized into M control volumes with uniform spacing $\Delta y = 1/M$. The temporal domain is discretized with a time step Δx. The current time is denoted by x^N and the center gridpoint is denoted by y_i:

$$\int_{x^N}^{x^{N+1}} \int_{V_i} \left(\frac{\partial u}{\partial x}\right) dV dx = \int_{x^N}^{x^{N+1}} \int_{V_i} 2\frac{\partial^2 u}{\partial y^2} dV dx + \int_{x^N}^{x^{N+1}} \int_{V_i} 2 dV dx.$$

This becomes:

$$(u_i^{N+1} - u_i^N)\Delta y = 2\left(\frac{u_{i+1}^{N+1} - 2u_i^{N+1} + u_{i-1}^{N+1}}{2(\Delta y)^2} + \frac{u_{i+1}^N - 2u_i^N + u_{i-1}^N}{2(\Delta y)^2}\right)\Delta y \Delta x + 2\Delta y \Delta x;$$

$$\implies \frac{u_i^{N+1} - u_i^N}{\Delta x} = \frac{1}{(\Delta y)^2}(u_{i+1}^{N+1} - 2u_i^{N+1} + u_{i-1}^{N+1} + u_{i+1}^N - 2u_i^N + u_{i-1}^N) + 2.$$

Rearranging to fit the linear system form $Au^{N+1} = b$, defining $r = \frac{\Delta x}{(\Delta y)^2}$:

$$-ru_{i-1}^{N+1} + (1+2r)u_i^{N+1} - ru_{i+1}^{N+1} = ru_{i-1}^N + (1-2r)u_i^N + ru_{i+1}^N + 2\Delta x.$$

This gives the tridiagonal elements of the A matrix for grid points in the center. Applying the boundary conditions with the half-cell approach to maintain cell-centered FVM consistency, the first and last rows of the A matrix instead use the form:

$$(1+6r)u_1^{N+1} - 2ru_2^{N+1} = (1-6r)u_1^N + 2ru_2^N + 2\Delta x \quad \text{(Left BC), and}$$

$$-2ru_{M-1}^{N+1} + (1+6r)u_M^{N+1} = 2ru_{M-1}^N + (1-6r)u_M^N + 2\Delta x \quad \text{(Right BC).}$$

The general flow of the numerical implementation is as follows:

1. Initialize parameters: M, Nt (number of time steps), Δy, Δx, and the solution array u with initial conditions.

2. Construct the A matrix based on the discretization scheme and boundary conditions.
3. For each time step, compute the right-hand side vector b, then solve for u^{N+1} using `scipy.linalg.solve_banded` (TDMA).
4. Make the previous solution u^{N+1} the current solution u^N and repeat the previous step until the desired final time is reached.

The implementation is done in Python, utilizing NumPy for array operations and SciPy for solving the linear system efficiently. The TDMA solver is an LU decomposition modification chosen for its suitability to the tridiagonal structure of the A matrix, ensuring computational efficiency and stability.

4 Presentation of Results

A 3D surface plot of the numerical solution $u(x, y)$ is shown below in Figure 1 to illustrate the evolution of the solution over time and space. The plot demonstrates the expected behavior of the solution, with the transient component decaying over time and the steady-state solution dominating as x increases. The system behavior closely resemebles the progression of a velocity profile in a lamniar channel or pipe. Comparison of the numerical solution to the analytical solution at various

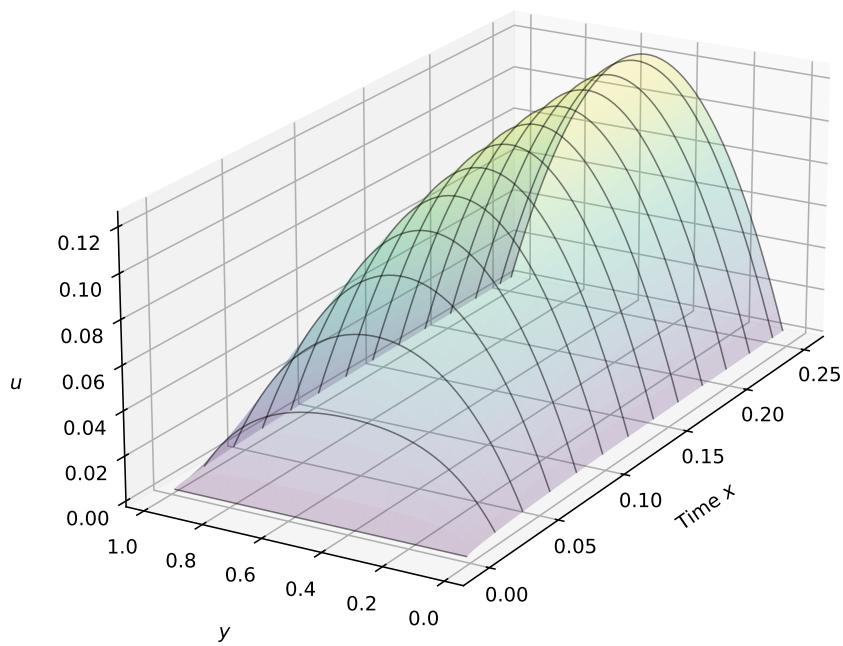
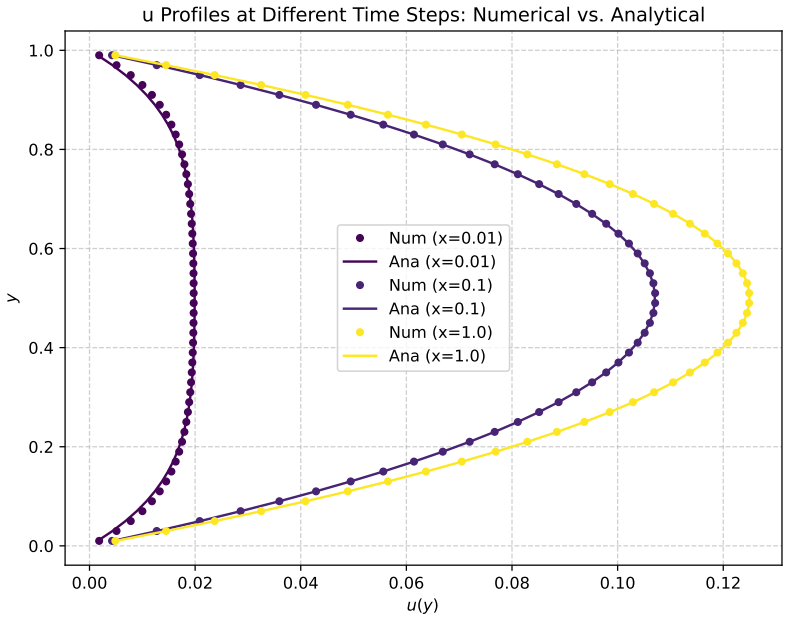
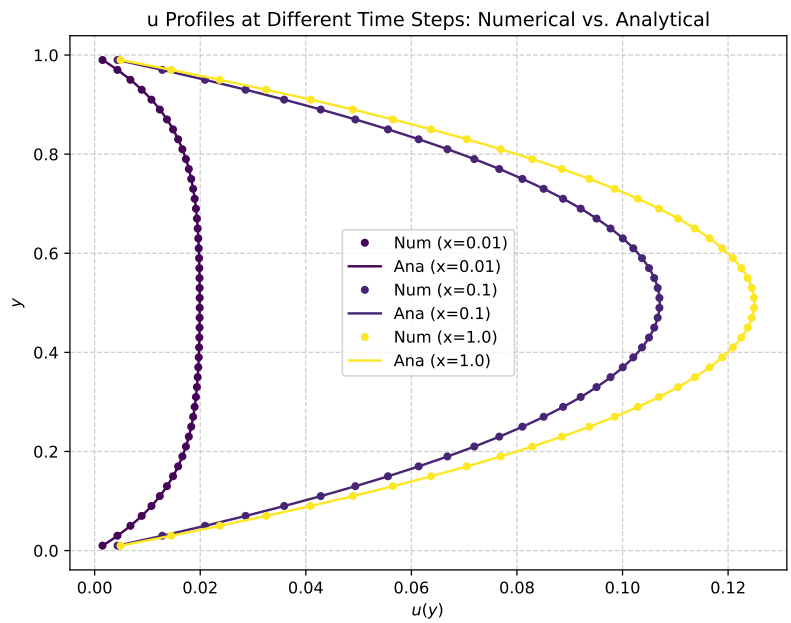


Figure 1: 3D Velocity Profile Development.

time steps is shown in Figure 2. The numerical solution closely matches the analytical solution, with discrepancies decreasing as the time step is reduced, demonstrating the convergence of the method.



(a) Large time step ($\Delta x = 0.01$).



(b) Small time step ($\Delta x = 0.001$).

Figure 2: Temporal resolution study.

5 Discussion

The numerical solution correctly obtains the asymptotic behavior of the solution, and the stability of this scheme and chosen grid resolution is demonstrated by:

1. The lack of oscillations in the solution sometimes observed in second order schemes even at varying time steps.
2. Preservation of the boundary conditions throughout the simulation.
3. Symmetry about the centerline $y = 0.5$ is maintained, consistent with the problem's symmetry.
4. The convergence of the numerical solution to the analytical solution as the time step is reduced, indicating that the method is consistent and stable.

6 Appendix: Code Listings

6.1 physics.py

```python
"""
physics.py
----------
This module defines the physical parameters of the system and models the analytical solution to the
parabolic equation being solved: du/dx - 2(d^2u/dy^2) = 2.
"""

import numpy as np

def get_bn_coefficient(n: int) -> float:
    """
    Computes the bn coefficient for the Fourier series solution to the PDE.

    Parameters
    ----------
    n : int
        The index of the Fourier coefficient to compute.

    Returns
    -------
    float
        The value of the bn coefficient for the given n, using the formula:
        B_n = [2 / (n*pi)^3] * [(-1)^n - 1]
    """
    if n % 2 == 0:
        return 0.0
    else:
        return 2.0 / (n * np.pi)**3 * ((-1)**n - 1)

def analytical_solution(y: np.ndarray, x: float, n_terms: int = 100) -> np.ndarray:
    """
```

```python
    Computes the analytical solution to the PDE at a given time x and spatial points y, using a Fourier series expansion.

    u(x,y) = SteadyState(y) + Transient(x,y)

    Parameters
    ----------
    y : np.ndarray
        The spatial points at which to evaluate the solution.
    x : float
        The time at which to evaluate the solution.
    n_terms : int, optional
        The number of terms in the Fourier series expansion to compute (default is 100).

    Returns
    -------
    np.ndarray
        The analytical solution evaluated at the given spatial points and time.
    """
    # 1. Steady state part: f(y) = y/2 - y^2/2
    steady_state = 0.5 * y * (1.0 - y)

    # 2. Transient part: Sum of B_n + sin(n*pi*y) * exp(-2*(n*pi)^2 * x)
    transient = np.zeros_like(y)
    for n in range(1, n_terms + 1):
        bn = get_bn_coefficient(n)
        eigenvalue = n * np.pi
        spatial_mode = np.sin(eigenvalue * y)
        temporal_decay = np.exp(-2.0 * eigenvalue**2 * x)

        transient += bn * spatial_mode * temporal_decay
    return steady_state + transient

def initial_condition(y: np.ndarray) -> np.ndarray:
    """
    Computes the initial condition for the PDE at time x=0, which is given by:
    u(0,y) = 0

    Parameters
    ----------
    y : np.ndarray
        The spatial points at which to evaluate the initial condition.

    Returns
    -------
    np.ndarray
        The initial condition at the given spatial points.
    """
    return np.zeros_like(y)
```

Listing 1: Physics module containing the analytical solution and coefficients.

6.2 solvers.py

```python
"""
solvers.py
```

```python
----------
This module assembles the tridiagonal matrix and sets up the linalg solver
for the 1D FVM discretization.
"""

import numpy as np
from scipy.linalg import solve_banded

def build_implicit_matrix(M: int, r: float) -> np.ndarray:
    """
    Constructs the LHS (implicit) matrix in banded format.

    Format for scipy.linalg.solve_banded:
    Row 0: Upper diagonal [0, c1, c2, ...]
    Row 1: Main diagonal [b1, b2, b3, ...]
    Row 2: Lower diagonal [a1, a2, a3, ..., 0]

    Inputs
    ------
    M : int
        Number of spatial grid points (control volumes).
    r : float
        The diffusion number, defined as r = (Delta x) / (Delta y^2).

    Returns
    -------
    np.ndarray
        A 3 x M array representing the banded matrix for the implicit scheme.
    """

    # Initialize a 3 x M array for the diagonals
    A = np.zeros((3, M))

    # 1. Main diagonal (Row 1)
    A[1, :] = 1.0 + 2.0 * r
    A[1, 0] = 1.0 + 6.0 * r # Left boundary correction
    A[1, -1] = 1.0 + 6.0 * r # Right boundary correction

    # 2. Upper diagonal (Row 0)
    A[0, 1:] = -r
    A[0, 1] = -2.0 * r # Left boundary correction

    # 3. Lower diagonal (Row 2)
    A[2, :-1] = -r
    A[2, -2] = -2.0 * r # Right boundary correction

    return A

def construct_rhs(u_n: np.ndarray, r: float, dx: float) -> np.ndarray:
    """
    Assembles the RHS vector 'b' for the system Au=b.

    Includes the explicit Crank-Nicolson terms and the sources term (2 * dx).

    Inputs
    ------
    u_n : np.ndarray
        The current time step values (M x 1).
    r : float
```

```python
        The diffusion number, defined as r = (Delta x) / (Delta y^2).
    dx : float
        The temporal spacing.

    Returns
    -------
    np.ndarray
        The RHS vector 'b' for the system Au=b.
    """
    M = len(u_n)
    b = np.zeros(M)
    source_term = 2.0 * dx

    # Interior nodes (i=1 to M-2 in 0-based indexing)
    for i in range (1, M - 1):
        b[i] = r * u_n[i - 1] + (1.0 - 2.0 * r) * u_n[i] + r * u_n[i + 1] + source_term

    # Left boundary (i=0)
    b[0] = (1.0 - 6.0 * r) * u_n[0] + 2.0 * r * u_n[1] + source_term

    # Right boundary (i=M-1)
    b[-1] = 2.0 * r * u_n[-2] + (1.0 - 6.0 * r) * u_n[-1] + source_term

    return b

def step_solver(A_banded: np.ndarray, b: np.ndarray) -> np.ndarray:
    """
    Solves the linear system Au=b for the next time step values.

    Inputs
    ------
    A_banded : np.ndarray
        The banded matrix representing the LHS of the system.
    b : np.ndarray
        The RHS vector for the system.

    Returns
    -------
    np.ndarray
        The solution vector for the next time step.
    """
    # (1, 1) indicates 1 lower and 1 upper diagonal
    return solve_banded((1, 1), A_banded, b)
```

Listing 2: Numerical solver implementation using the Crank-Nicolson scheme.

6.3 post.py

```python
"""
post.py
-------
This module contains functions for post-processing the numerical solution, including error analysis and visualization.
Its purpose is to compare the numerical solution obtained from the finite volume method with the analytical solution derived from the Fourier series expansion.
"""
```

```python

import numpy as np
import matplotlib.pyplot as plt

def calculate_error(u_numerical: np.ndarray, u_analytical: np.ndarray) -> float:
    """
    Computes the L2 norm of the error between the numerical and analytical solutions.

    Inputs
    ------
    u_numerical : np.ndarray
        The numerical solution array at the final time step.
    u_analytical : np.ndarray
        The analytical solution array evaluated at the same spatial points and final time.

    Returns
    -------
    float
        The L2 norm of the error, defined as sqrt(mean((u_numerical - u_analytical)^2)).
    """
    error = np.sqrt(np.mean((u_numerical - u_analytical)**2))
    return error

def plot_profiles(y: np.ndarray,
                  profiles: list[tuple[float, np.ndarray, np.ndarray]],
                  save_path: str = "u_profiles.pdf"):
    """
    Generates a comparison plot for u profiles at different x-stations.

    Inputs
    ------
    y : np.ndarray
        The spatial grid points (y-coordinates).
    profiles : list[tuple[float, np.ndarray, np.ndarray]]
        A list of tuples (x_position, u_numerical, u_analytical) for each profile to be plotted.
    save_path : str
        The file path where the plot will be saved.
    """
    plt.figure(figsize=(8, 6))

    # Iterate through the saved stations
    for x_val, u_num, u_ana in profiles:
        color = plt.cm.viridis(x_val / profiles[-1][0]) # Color based on x-progress

        # Plot numercial as points and analytical as a line
        plt.plot(u_num, y, 'o', markersize=4, label=f'Num (x={x_val})', color=color)
        plt.plot(u_ana, y, '-', linewidth=1.5, label=f'Ana (x={x_val})', color=color)

    plt.xlabel(r'$u(y)$')
    plt.ylabel(r'$y$')
    plt.title('u Profiles at Different Time Steps: Numerical vs. Analytical')
    plt.grid(True, linestyle='--', alpha=0.6)
```

```python
    plt.legend()

    # Save with high resolution for the report
    plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.show()
    print(f"Plot saved successfully to {save_path}")

def plot_convergence(dx_list: list[float], errors: list[float]):
    """
    Plots error vs step size on a log-log scale to check for 2nd order accuracy.

    Inputs
    ------
    dx_list : list[float]
        A list of step sizes (Delta x) used in the convergence study.
    errors : list[float]
        A list of corresponding L2 errors for each step size.
    """
    plt.figure(figsize=(6, 5))
    plt.loglog(dx_list, errors, '-ok', label='Computed Error')

    # Reference line for 2nd order slope (Error proportional to dx^2)
    plt.loglog(dx_list, [dx**2 for dx in dx_list], '--', label='2nd Order Reference')

    plt.xlabel(r'Step Size $\Delta x$')
    plt.ylabel('L2 Error')
    plt.legend()
    plt.grid(True, which="both", ls="-", alpha=0.5)
    plt.show()


from mpl_toolkits.mplot3d import Axes3D
from matplotlib import cm

def plot_3d_evolution(y: np.ndarray,
                      all_results: list,
                      x_range: tuple = (0.0, 0.2),
                      save_path: str = "isometric_flow.pdf"):
    """
    Creates a 3D isometric plot of u(y) evolving over time (x) to visualize the flow development.

    Inputs
    ------
    y : np.ndarray
        The spatial grid points (y-coordinates).
    all_results : list
        A list of tuples (x_position, u_profile) for each time step saved during the simulation.
    x_range : tuple
        The range of x (time) to include in the plot, defined as (x_min, x_max).
    save_path : str
        The file path where the plot will be saved.
    """

    fig = plt.figure(figsize=(12, 8))
    ax = fig.add_subplot(111, projection='3d')

```

```python
    # --- 1. Filter the X-Window ---
    # Only keep data where x is within the user-defined x_range
    filtered = [res for res in all_results if x_range[0] <= res[0] <= x_range[1]]

    x_coords = np.array([res[0] for res in filtered])
    u_values = np.array([res[1] for res in filtered])
    X, Y = np.meshgrid(x_coords, y)
    Z = u_values.T

    # --- 2. The "Skin" (Transparent Surface) ---
    ax.plot_surface(X, Y, Z, cmap='viridis', alpha=0.2, linewidth=0, antialiased=True)

    # --- 3. The "Skeleton" (Progression of Curves) ---
    # cstride=0 means no lines along the y-direction (only x-stations)
    # rstride controls how many x-curves are drawn
    ax.plot_wireframe(X, Y, Z, color='black', alpha=0.5, rstride=0, cstride=20, linewidth=0.8)

    # --- 4. Aesthetics ---
    ax.view_init(elev=20, azim=-150)
    ax.set_box_aspect((2, 1, 0.8)) # Stretch the x-axis for a "tunnel" feel

    ax.set_xlabel('Time $x$', labelpad=15)
    ax.set_ylabel('$y$', labelpad=15)
    ax.set_zlabel('$u$', labelpad=10)

    plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.show()
```

Listing 3: Post-processing and visualization tools.

6.4 main.py

```python
"""
Ben Rees
02/22/2026
CFD Project 1
main.py
-------
Main execution script for the 1D Channel Flow development simulation.
Solves the parabolic momentum equation using a Crank-Nicolson FVM scheme.

Case Parameters:
- Domain: y in [0, 1]
- Scheme: Crank-Nicolson (2nd order)
- Linear Solver: TDMA (via scipy.linalg.solve_banded)
"""

import numpy as np
import physics as phys
import solvers as sol
import post

def run_simulation(M: int = 50, x_final: float = 1.0, dx: float = 0.001):
    # 1. Grid Generation
    dy = 1.0 / M
```

```python
    y = np.linspace(dy/2, 1.0 - dy/2, M) # Cell-centered nodes
    r = dx / (dy**2)

    # 2. Initialization
    u = phys.initial_condition(y)
    x = 0.0

    # Stations to save for plotting (as requested by the report)
    save_stations = [0.01, 0.1, 1.0]
    results = []

    # NEW: Storage for the "Crazy" 3D Plot
    all_history = []
    save_interval = 20  # Save every 20 steps to keep the 3D plot smooth but fast
    step_count = 0

    # 3. Pre-compute constant Implicit Matrix
    A_banded = sol.build_implicit_matrix(M, r)

    # 4. Time-Marching Loop
    print(f"Starting simulation with r={r:.4f}...")
    while x < x_final:
        # Check if we should save current profile before stepping
        for station in save_stations:
            if abs(x - station) < dx/2:
                u_ana = phys.analytical_solution(y, x)
                results.append((round(x, 2), u.copy(), u_ana))

        #Save data for 3D plot every 'save_interval' steps
        if step_count % save_interval == 0:
            all_history.append((x, u.copy()))

        # Build RHS and Solve
        b = sol.construct_rhs(u, r, dx)
        u = sol.step_solver(A_banded, b)

        x += dx

    # 5. Final Comparison & Plotting
    post.plot_profiles(y, results, save_path="u_comparison.pdf")

    # Plot 2: The "Portfolio" Plot (3D)
    post.plot_3d_evolution(y, all_history, x_range=(0.0, 0.25), save_path="isometric_flow.pdf")

    # Final error check
    u_final_ana = phys.analytical_solution(y, x_final)
    error = post.calculate_error(u, u_final_ana)
    print(f"Simulation complete. L2 Error at x=1.0: {error:.2e}")

if __name__ == "__main__":
    # Standard parameters for the project
    run_simulation(M=50, x_final=1.05, dx=0.001)
```

Listing 4: Main script to run the simulation and generate results.

7 Code Availability

All Python scripts are available on GitHub at: https://github.com/Ben-JaminRees/Fuel-Cell-Course-Spring tree/main/Assignment_2_2.